Autor: Iván García Santillán

Algoritmos predictivos de clasificación: Vecinos más cercanos (K-NN)

Introducción

El algoritmo k-Nearest Neighbors (k-NN) es un método de aprendizaje automático supervisado utilizado para problemas de clasificación y regresión. Éste tiene diversas aplicaciones prácticas en campos como el análisis de datos, aprendizaje automático y sistemas de recomendación.

En medicina, se utiliza para clasificar pacientes en categorías de riesgo basándose en características similares a casos conocidos.

En el sector financiero, ayuda en la detección de fraudes identificando patrones de transacciones sospechosas similares a casos previos de fraude.

En el comercio minorista y en servicios de *streaming*, k-NN es esencial para los sistemas de recomendación, sugiriendo productos, películas o música basándose en las preferencias de usuarios con gustos similares.

También es utilizado en la agricultura para la clasificación y diagnóstico de enfermedades de plantas, y en la robótica para la navegación y clasificación de obstáculos. La simplicidad y efectividad del algoritmo lo hacen versátil para aplicaciones que requieren clasificación o recomendación basada en similitudes de datos.

Aunque también se debe considerar algunas desventajas como una posible baja de la precisión comparado con otros algoritmos y un uso de memoria moderado dependiendo del tamaño del conjunto de datos utilizado.

A continuación, se detallan el funcionamiento del algoritmo y un ejemplo en Python, Keras y TensorFlow para la predicción del cáncer de mama utilizando un data set público.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Contenido

La idea fundamental detrás del algoritmo k-NN es que los puntos de datos similares tienden a estar cerca unos de otros en el espacio de características. Por lo tanto, para predecir una nueva instancia, el algoritmo busca los "k" puntos de datos más cercanos en el conjunto de entrenamiento y toma una decisión basada en la mayoría de las etiquetas de clase (en el caso de clasificación) o en el promedio (en el caso de regresión) de las etiquetas de los vecinos más cercanos.

Algunos conceptos clave relacionados con el algoritmo k-NN son los siguientes:

Parámetro k: el valor de "k" en k-NN representa el número de vecinos más cercanos que se deben considerar al hacer una predicción. Un valor pequeño de "k" (como 1) puede llevar a predicciones ruidosas y sensibles a valores atípicos, mientras que un valor grande de "k" suaviza las predicciones pero puede perder detalles finos en los datos. La elección del valor de "k" es crítica en k-NN y puede afectar significativamente el rendimiento del modelo.

Función de Distancia: para determinar qué puntos de datos son los más cercanos, se utiliza una función de distancia, como la distancia euclidiana, la distancia de Manhattan o la distancia de Minkowski. La elección de la función de distancia depende del problema y los datos.

Normalización: antes de aplicar k-NN, es importante normalizar las características para que todas tengan la misma influencia en la distancia entre puntos. Esto es especialmente importante cuando las características tienen escalas muy diferentes.

El algoritmo k-NN es simple de entender y de implementar, y es útil en una variedad de aplicaciones, como clasificación de documentos, recomendación de productos, diagnóstico médico y más. Sin embargo, es computacionalmente costoso en conjuntos de datos grandes, ya que requiere calcular la distancia entre la nueva instancia y todos los puntos de datos de entrenamiento.

A continuación, se presenta un ejemplo del algoritmo K-NN en Python, Keras y TensorFlow para la predicción del cáncer de mama (benigno=0, maligno=1)

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

utilizando el conjunto de datos Breast Cancer Wisconsin (Diagnostic) disponible en la librería scikit-learn.

```
# K-NN for breast cancer

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import classification_report
from sklearn.neighbors import KNeighborsClassifier

# Carga el conjunto de datos Breast Cancer
dataset = load_breast_cancer()
X = dataset.data # 569x30
y = dataset.target # 569x1

# Divide el conjunto de datos en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Normaliza los datos para que todas las características tengan una escala similar
scaler = MinMaxScaler(feature_range=(0,1)) # [0, 1]
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Crea y entrena el modelo K-NN
model = KNeighborsClassifier(n_neighbors=3, p=2, # Función euclidean weights='distance')

model.fit(X_train, y_train)

# Realiza predicciones usando el conjunto de prueba
y_pred = model.predict(X_test)

# Convierte las probabilidades en etiquetas binarias (0 o 1)
y_pred = (y_pred > 0.5)
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

```
# Muestra el informe de evaluación del modelo entrenado
print(classification_report(y_test, y_pred))

# Matriz de confusión:
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)
print("confusion matrix: \n", cm)
# gráfica de la CM
plt.figure(figsize = (8,4))
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel('Prediction', fontsize = 12)
plt.ylabel('Real', fontsize = 12)
plt.show()

# Exactitud:
from sklearn.metrics import accuracy_score
acc = accuracy_score(y_test, y_pred)
print("accuracy: ", acc)

# Sensibilidad:
from sklearn.metrics import recall_score
recall = recall_score(y_test, y_pred)
print("recall: ", recall)

# Precisión:
from sklearn.metrics import precision_score
precision = precision_score(y_test, y_pred)
print("precision: ", precision)

# Especificidad
# 'specificity' is just a special case of 'recall'.
# specificity is the recall of the negative class
specificity = recall_score(y_test, y_pred, pos_label=0)
print("specificity: ", specificity)
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

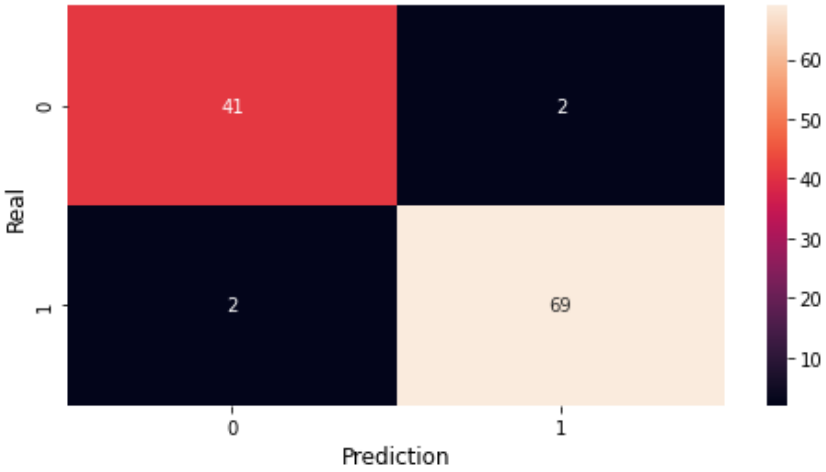
```
# Puntuación F1:
from sklearn.metrics import f1_score
f1 = f1_score(y_test, y_pred)
print("f1 score: ", f1)

# Área bajo la curva:
from sklearn.metrics import roc_auc_score
auc = roc_auc_score(y_test, y_pred)
print("auc: ", auc)

# Curva ROC
from sklearn.metrics import roc_curve
plt.figure()
lw = 2
plt.plot(roc_curve(y_test, y_pred)[0], roc_curve(y_test, y_pred)[1],
color='darkorange',lw=lw, label='ROC curve (area = %0.2f)' %auc)
plt.plot([0, 1], [0, 1], color='navy', lw=lw, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic')
plt.legend(loc="lower right")
plt.show()

# R Score (R^2 coefficient of determination)
from sklearn.metrics import r2_score
R = r2_score(y_test, y_pred)
print("R2: ", R)
```

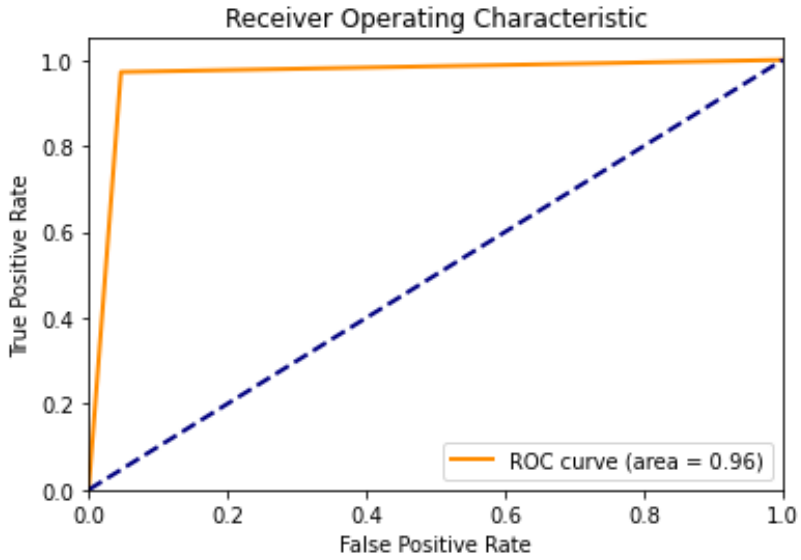
Los resultados del modelo K-NN se presentan a continuación:

```
accuracy:  0.96
recall:  0.97
precision:  0.97
specificity:  0.95
f1 score:  0.97
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

auc: 0.96
R2: 0.85





Un clasificador adecuado es aquel que alcanza una exactitud (*accuracy*) superior al 85 %. Por lo que, este modelo K-NN entrenado cumple con este criterio de calidad.

Adicionalmente, tenga en cuenta que se pueden ajustar varios parámetros del algoritmo (como *n_neighbors, p, weights*, etc.) para obtener un modelo óptimo.

Revise la documentación (ayuda en línea https://scikit-learn.org/stable/supervised_learning.html) del algoritmo para más detalles y experimente con ellos.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Finalmente, aunque K-NN es un modelo fundamental en la caja de herramientas de análisis predictivo, como cualquier otra técnica, presenta tanto ventajas como desventajas que se resumen a continuación:

Ventajas de K-NN:

Fácil de entender e implementar: Una de las mayores ventajas de K-NN es su simplicidad. El algoritmo es intuitivo y fácil de implementar, y no requiere entrenamiento explícito, lo que lo convierte en un método perezoso (*lazy*).

Flexible a la función de la distancia: K-NN puede trabajar con cualquier función de distancia, lo que lo hace adecuado para contextos donde la noción de distancia es no estándar o muy específica (como distancia Euclidiana, Manhattan, Minkowski, o incluso medidas personalizadas).

Eficaz con un gran número de clases: K-NN puede ser eficaz cuando hay un gran número de posibles etiquetas de salida; su capacidad para diferenciar no está limitada por las características de los datos.

Desventajas de K-NN:

Alto costo computacional: K-NN puede ser muy lento en bases de datos grandes porque cada nueva predicción requiere un cálculo de distancia a cada uno de los puntos de entrenamiento, lo cual puede ser computacionalmente costoso.

Alto uso de memoria: dado que K-NN es un algoritmo perezoso (*lazy*) y no genera un modelo explícito, necesita almacenar todo el conjunto de datos de entrenamiento para hacer predicciones, lo que puede resultar en un alto uso de memoria.

Sensible a variables irrelevantes: K-NN puede ser muy sensible a características irrelevantes o redundantes porque todas las características contribuyen aproximadamente por igual a la distancia entre las muestras.

Sensible a datos no balanceados: en un conjunto de datos donde las clases no están equitativamente representadas, K-NN puede estar sesgado hacia las clases más frecuentes.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Dificultad con clases *overlapping*: K-NN puede tener dificultades cuando los límites de decisión entre clases son borrosos. Su desempeño puede degradarse significativamente si las clases se superponen considerablemente.

Efecto de la maldición de la dimensionalidad: a medida que el número de características aumenta, el espacio de características se vuelve exponencialmente grande, lo que diluye la efectividad de la distancia euclidiana y puede hacer que K-NN actúe de manera subóptima.

Conclusiones

En esta lectura se ha revisado los fundamentos del algoritmo k-Nearest Neighbors (k-NN) para abordar problemas de clasificación binaria utilizando Python y TensorFlow-Keras. K-NN es un algoritmo potente y flexible que se adapta bien a una variedad de problemas de clasificación y regresión, es crucial tener en cuenta sus limitaciones, especialmente en términos de eficiencia y rendimiento en grandes volúmenes de datos o datos con muchas características. A menudo, técnicas como la reducción de dimensiones, la selección de características, y el ajuste del número de vecinos (k) son necesarias para mejorar su eficacia y eficiencia.

Por lo que, se motiva a los estudiantes a revisar material y ejemplos adicionales disponibles libremente en Internet para profundizar en la comprensión del algoritmo y su aplicación eficaz en diferentes problemas de variada complejidad.

Bibliografía

Berzal, Fernando (2019). *Redes neuronales & Deep Learning*. USA: Independently published.

Gerón, A. (2023). *Aprende Machine Learning con Scikit-Learn, Keras y TensorFlow: Conceptos, herramientas y técnicas para conseguir sistemas inteligentes.* Tercera Edición. Anaya Multimedia.

Torres, J. (2020). *Python Deep Learning: Introducción práctica con Keras y TensorFlow 2*. Marcombo.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Autor: Iván García Santillán

Algoritmos predictivos de clasificación: Regresión logística

Introducción

Es un método de aprendizaje automático supervisado utilizado para resolver problemas de clasificación binaria o multinomial (más de dos clases). Aunque su nombre incluye la palabra "regresión", en realidad se trata de un algoritmo de clasificación.

Se utiliza comúnmente en una variedad de aplicaciones, como la detección de *spam* de correo electrónico, diagnósticos médicos, clasificación de documentos, entre otros. Por ejemplo, en medicina, es fundamental para predecir la probabilidad de ocurrencia de ciertas enfermedades, como la diabetes o el cáncer, basándose en factores de riesgo.

En el sector financiero, se aplica en la evaluación del riesgo crediticio, determinando la probabilidad de que un cliente incumpla con sus pagos.

En *marketing*, se utiliza para predecir la probabilidad de que un cliente compre un producto o responda a una campaña específica. Además, la regresión logística es esencial en el campo de la investigación social y política para analizar la influencia de diversas variables sobre resultados binarios, como la decisión de votar por un candidato.

Su capacidad para proporcionar probabilidades y su facilidad de interpretación la hacen una herramienta valiosa para la toma de decisiones basada en datos. Aunque también se deben considerar algunas desventajas como la sensibilidad a datos atípicos (*outliers*), afectando significativamente la estimación de los coeficientes y distorsionando el modelo. Además, el problema de la multicolinealidad donde hay alta correlación entre las variables independientes (predictoras), así como el problema con datos desbalanceados donde puede desarrollar un sesgo hacia la clase más frecuente.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

A continuación, se detallan el funcionamiento del algoritmo y un ejemplo en Python, Keras y TensorFlow para la predicción del cáncer de mama utilizando un data set público.

Contenido

El nombre de regresión logística proviene de la función **logística** (también conocida como función **sigmoide**) que utiliza para modelar la relación entre las variables de entrada (características) y la probabilidad de que una instancia pertenezca a una clase específica, como se indica en la Figura 16.

Figura 16. Función logística (sigmoide)

$$f(x) = \frac{1}{1+e^{-x}}$$

La función logística tiene la siguiente forma:

$$P(Y=1) = \frac{1}{1+e^{-(a_0+a_1X_1+a_2X_2+\ldots+a_nX_n)}}$$

Donde:

- *P(Y=1)* es la probabilidad de pertenecer a la clase 1.
- *e* es la base del logaritmo natural.
- a_0, a_1, a_2, …, a_n son coeficientes que se aprenden (ajustan) durante el entrenamiento.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

- $X_1, X_2, \ldots, X_n$ son las variables de entrada.

Durante la fase de entrenamiento, el algoritmo de regresión logística ajusta los coeficientes $a_0, a_1, a_2, \ldots, a_n$ de manera que el modelo se ajuste mejor a los datos de entrenamiento.

Una vez que el modelo está entrenado, se puede utilizar para hacer predicciones sobre nuevos datos, es decir, el modelo calcula la probabilidad de que esa instancia pertenezca a una de las clases. Si la probabilidad calculada es mayor que el umbral (generalmente 0.5), se clasifica como la clase positiva (1); de lo contrario, se clasifica como la clase negativa (0) en un problema de clasificación binaria.

A continuación, se presenta un ejemplo del algoritmo de regresión logística en Python, Keras y TensorFlow para la predicción del cáncer de mama (benigno=0, maligno=1) utilizando el conjunto de datos Breast Cancer Wisconsin (Diagnostic) disponible en la librería scikit-learn.

```
# Logistic regression for breast cancer
# importa librerías necesarias
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report
from sklearn.linear_model import LogisticRegression

# Carga el conjunto de datos Breast Cancer
dataset = load_breast_cancer()
X = dataset.data # 569x30
y = dataset.target # 569x1

# Divide el conjunto de datos en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Normaliza los datos para que tengan una escala similar
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

```
X_test = scaler.transform(X_test)

# Crea y entrena el modelo de regresión logistica
model = LogisticRegression(multi_class='auto', solver='lbfgs', max_iter=100)
model.fit(X_train, y_train)

# Realiza predicciones usando el conjunto de prueba
y_pred = model.predict(X_test)

# Convierte las probabilidades en etiquetas binarias (0 o 1)
y_pred = (y_pred > 0.5)

# Muestra el informe de evaluación del modelo entrenado
print(classification_report(y_test, y_pred))

# Matriz de confusión:
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)
print("confusion matrix: \n", cm)
# gráfica de la CM
plt.figure(figsize = (8,4))
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel('Prediction', fontsize = 12)
plt.ylabel('Real', fontsize = 12)
plt.show()

# Exactitud:
from sklearn.metrics import accuracy_score
acc = accuracy_score(y_test, y_pred)
print("accuracy: ", acc)

# Sensibilidad:
from sklearn.metrics import recall_score
recall = recall_score(y_test, y_pred)
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

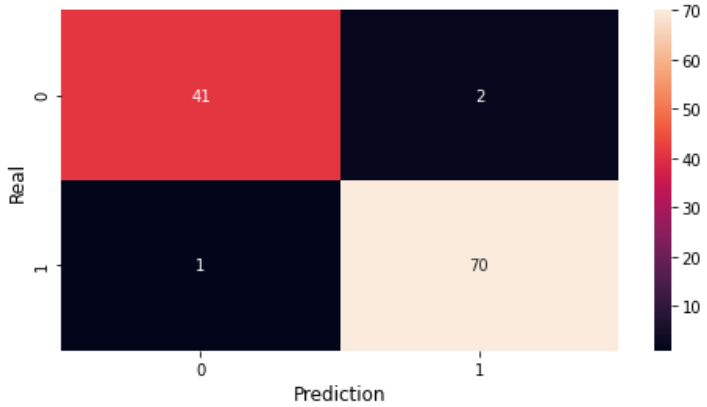
```
print("recall: ", recall)

# Precisión:
from sklearn.metrics import precision_score
precision = precision_score(y_test, y_pred)
print("precision: ", precision)

# Especificidad
# 'specificity' is just a special case of 'recall'.
# specificity is the recall of the negative class
specificity = recall_score(y_test, y_pred, pos_label=0)
print("specificity: ", specificity)

# Puntuación F1:
from sklearn.metrics import f1_score
f1 = f1_score(y_test, y_pred)
print("f1 score: ", f1)

# Área bajo la curva:
from sklearn.metrics import roc_auc_score
auc = roc_auc_score(y_test, y_pred)
print("auc: ", auc)

# Curva ROC
from sklearn.metrics import roc_curve
plt.figure()
lw = 2
plt.plot(roc_curve(y_test, y_pred)[0], roc_curve(y_test, y_pred)[1],
color='darkorange',lw=lw, label='ROC curve (area = %0.2f)' %auc)
plt.plot([0, 1], [0, 1], color='navy', lw=lw, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic')
plt.legend(loc="lower right")
plt.show()
```

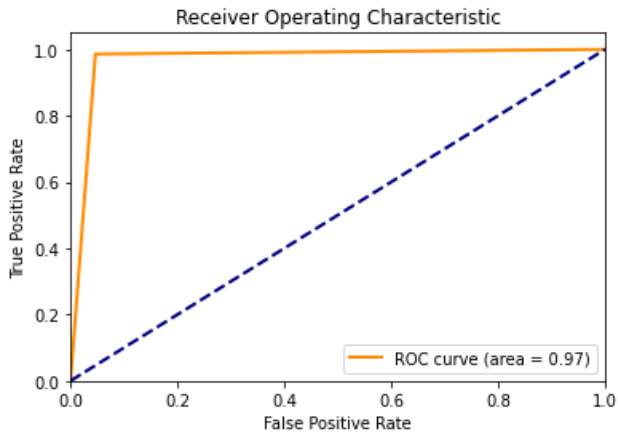
Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

```
# R Score (R^2 coefficient of determination)
from sklearn.metrics import r2_score
R = r2_score(y_test, y_pred)
print("R2: ", R)
```

Los resultados del modelo de regresión logística se presentan a continuación:

accuracy: 0.97
recall: 0.98
precision: 0.97
specificity: 0.95
f1 score: 0.97
auc: 0.97
R2: 0.88





Un clasificador adecuado es aquel que alcanza una exactitud (*accuracy*) superior al 85 %. Por lo que, este modelo de regresión logística entrenado cumple con este criterio de calidad.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Adicionalmente, tenga en cuenta que se pueden ajustar varios parámetros del algoritmo (como multi_class, solver, max_iter, etc.) para obtener un modelo óptimo. Revise la documentación (ayuda en línea https://scikit-learn.org/stable/supervised_learning.html) del algoritmo para más detalles y experimente con ellos.

Finalmente, aunque la regresión logística es un modelo fundamental en la caja de herramientas de análisis predictivo, como cualquier otra técnica, presenta tanto ventajas como desventajas que se resumen a continuación:

Ventajas de la Regresión Logística:

Interpretabilidad: una de las principales fortalezas de la regresión logística es su interpretabilidad fácil y directa. Los coeficientes del modelo pueden ser interpretados para cada uno de los predictores independientes, proporcionando una clara comprensión del impacto de cada variable en la probabilidad del resultado.

Eficiencia computacional: la regresión logística es menos intensiva en términos computacionales en comparación con modelos más complejos como las redes neuronales, lo que la hace relativamente rápida de entrenar.

Buen rendimiento con variables categóricas y numéricas: el algoritmo funciona bien con una combinación de características numéricas y categóricas.

Probabilidades de salida: a diferencia de muchos otros clasificadores, la regresión logística proporciona una probabilidad cuantitativa para las predicciones, lo que puede ser crucial para ciertas decisiones donde el umbral de decisión necesita ser ajustado.

Bueno para modelos lineales: funciona muy bien si la relación entre la variable dependiente y las independientes es aproximadamente lineal, común en problemas de complejidad baja y media.

Desventajas de la Regresión Logística:

No es adecuada para relaciones no lineales: el algoritmo no puede manejar relaciones no lineales sin transformación de las variables independientes,

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

común en problemas complejos. El término modelar “relaciones no lineales" se refiere a la capacidad de un modelo estadístico o de aprendizaje automático para capturar y representar relaciones entre variables que no son simplemente proporcionales o directas. En una relación no lineal, los cambios en una variable no producen cambios proporcionales y predecibles en otra variable, lo que significa que la relación entre variables no puede ser descrita adecuadamente por una línea recta, sino por otro tipo de relación como cuadrática, exponencial, logarítmica, sinusoidal, etc.

Sensibilidad a datos no balanceados: si las clases no están aproximadamente igualmente representadas, la regresión logística puede desarrollar un sesgo hacia la clase dominante. Esto puede requerir técnicas de muestreo o ponderación para corregir.

Influencia de valores atípicos: como en muchos modelos lineales, los valores atípicos pueden tener un efecto desproporcionadamente alto en la estimación del modelo, lo que puede llevar a estimaciones y predicciones pobres.

Multicolinealidad: problemas de multicolinealidad pueden surgir cuando dos o más variables predictoras son altamente correlacionadas. Ésto no sólo influye en la precisión de la estimación de los coeficientes, sino también en la interpretación de estos.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Conclusiones

En esta lectura se ha revisado los fundamentos del algoritmo de regresión logística para abordar problemas de clasificación binaria utilizando Python y TensorFlow-Keras.

La regresión logística es una herramienta poderosa y flexible para problemas de clasificación, especialmente cuando la claridad de la interpretación y la velocidad de cálculo son preocupaciones primordiales. Sin embargo, su aplicación debe ser considerada viendo que las limitaciones del modelo no afecten negativamente los resultados del análisis. Es importante experimentar y ajustar varios parámetros del algoritmo para obtener un modelo óptimo.

Por lo que, se motiva a los estudiantes a revisar material y ejemplos adicionales disponibles libremente en Internet para profundizar en la comprensión del algoritmo y su aplicación eficaz en diferentes problemas de variada complejidad.

Bibliografía

Berzal, Fernando (2019). *Redes neuronales & Deep Learning*. USA: Independently published.

Gerón, A. (2023). *Aprende Machine Learning con Scikit-Learn, Keras y TensorFlow: Conceptos, herramientas y técnicas para conseguir sistemas inteligentes.* Tercera Edición. Anaya Multimedia.

Torres, J. (2020). *Python Deep Learning: Introducción práctica con Keras y TensorFlow 2*. Marcombo.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Autor:
Iván García Santillán

Herramientas y conjunto de datos para Ciencia de Datos

Introducción

En esta unidad se utiliza herramientas y conjuntos de datos públicos para el entrenamiento de algunos algoritmos de aprendizaje automático supervisado (clasificación) siguiendo una metodología para el desarrollo de proyectos de análisis de datos y los aspectos éticos en el uso y aprovechamiento del Big Data. Los algoritmos de clasificación que abordaremos aquí son regresión logística, y vecinos más cercanos (K-NN) para problemas de clasificación binaria y/o multi-clasificación, todos ellos utilizando el lenguaje Python y el framework TensorFlow-Keras.

Palabras clave

Herramientas para Data Science, metodologías, datasets, aprendizaje automático.

Reto

¿Qué herramientas y algoritmos resultan más adecuados para abordar un problema de clasificación binario/multiclase?

Desarrollo

2. Herramientas para ciencia de datos

2.1. Introducción al aprendizaje supervisado y no supervisado

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

El aprendizaje supervisado y no supervisado son dos enfoques fundamentales en el campo del aprendizaje automático.

En el **aprendizaje supervisado**, se utilizan datos de entrenamiento que están etiquetados, lo que significa que cada ejemplo en el conjunto de datos tiene una etiqueta o respuesta conocida. El aprendizaje supervisado se utiliza típicamente para desarrollar modelos predictivos. Esto incluye modelos de clasificación (como predecir si un correo electrónico es *spam* o no) y modelos de regresión (como predecir el precio de una casa en función de sus características).

En el **aprendizaje no supervisado**, los datos de entrenamiento no están etiquetados. El modelo debe encontrar patrones, estructuras o agrupamientos inherentes en los datos sin conocer las respuestas correctas de antemano. El aprendizaje no supervisado se utiliza para técnicas descriptivas que ayudan a entender mejor la estructura y las relaciones en los datos. Esto incluye el *clustering* (para agrupar datos similares) y la reducción de dimensionalidad (para representar datos en un espacio de menor dimensión).

Ambos enfoques, supervisado y no supervisado, son fundamentales en la ciencia de datos y el aprendizaje automático, y se utilizan en una variedad de aplicaciones según los objetivos de modelado y los datos disponibles.

Técnicas de análisis de datos masivos

Técnicas predictivas:

- **Clasificación:** el atributo a predecir *(target)* es de tipo cualitativo (categoría). Ej. clasificación de objetos en imágenes: perro, gato, avión, *laptop*. Algoritmos: RNA (MLP), SVM, decision tree, K-NN, Naive Bayes, logistic regression, Random Forest (ensemble).

- **Regresión:** similar a la clasificación, pero el atributo a predecir es de tipo cuantitativo. Algoritmos: linear and multiple regression with least-squares method, RNA, Regression tree, SVM, Random Forest.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

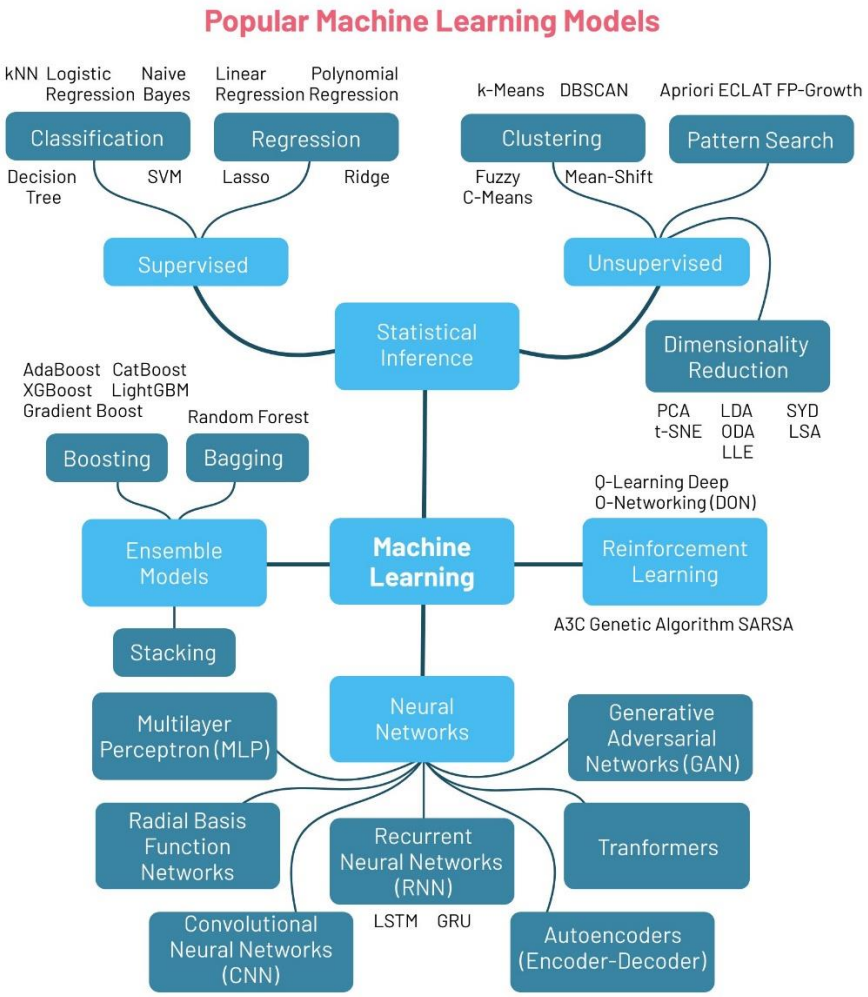
Técnicas Descriptivas:

- **Agrupamiento:** segmentación en grupos homogéneos. Ej. clientes A, B, C. Algoritmos: k-means, Expectation Maximization (EM), K-NN.

- **Asociación:** identificación de productos que habitualmente se compran juntos (análisis de la canasta). Ej. pan, azúcar -> leche. Reglas antecedente-consecuente. Algoritmos: A priori, FP Growth.

En la Figura 11 se muestra los tipos de modelos de machine learning existentes en la literatura.

Figura 11. Tipos de modelos de machine learning



Fuente: ML4Devs (2020)

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

En lo que sigue de la unidad se revisarán ciertos algoritmos predictivos de clasificación.

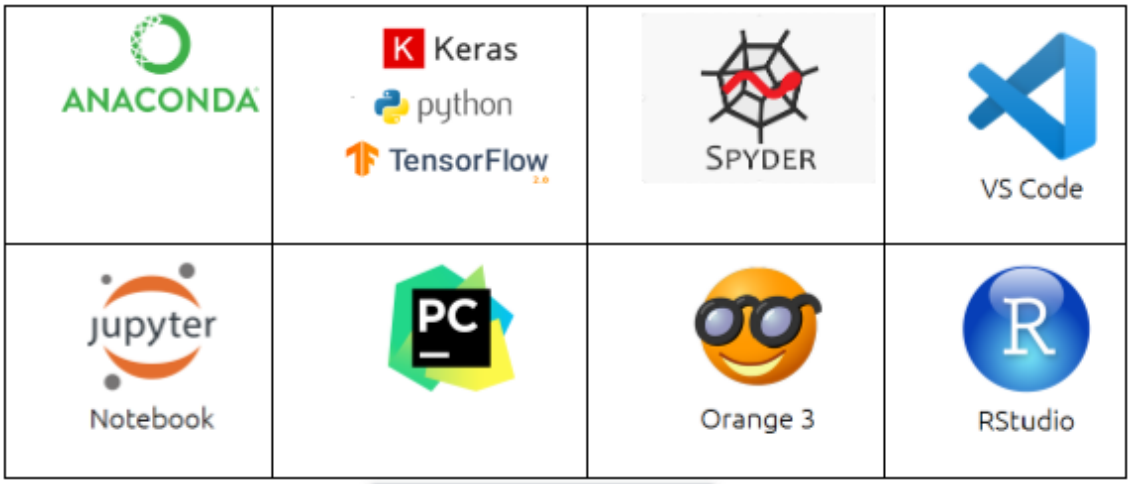
2.2. Configuración del entorno de Python, Keras y TensorFlow

Siga el tutorial disponible aquí, para configurar localmente un entorno de desarrollo de aprendizaje automático utilizando la herramienta **Anaconda Navigator**. Anaconda es una distribución libre y abierta de los lenguajes Python y R, utilizada en ciencia de datos, y aprendizaje automático que permite instalar y administrar paquetes, dependencias y entornos para las ciencias de datos con Python de una manera muy sencilla. Ayuda a desarrollar proyectos de ciencia de datos utilizando diversos entornos de desarrollo como Jupyter, PyCharm, Spyder, VS Code, Orange, RStudio, etc., como se illustran en la Figura 12.

En este tutorial se cubrirá los siguientes pasos:

- Descargar Anaconda navigator
- Instalar Anaconda
- Iniciar y actualizar Anaconda
- Actualizar la biblioteca scikit-learn
- Instalar bibliotecas de aprendizaje profundo

Figura 12. Entornos de desarrollo disponibles en Anaconda Navigator



Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

2.3. Uso de la GPU en TensorFlow

Siga el tutorial disponible aquí, para configurar localmente la GPU y sacar provecho del *hardware* disponible en su PC/laptop para el entrenamiento de modelos de datos.

En este tutorial se cubrirá los siguientes pasos:

- Configuración
- Descripción general
- Ubicación del dispositivo de registro
- Colocación manual de dispositivos
- Limitar el crecimiento de la memoria de la GPU
- Uso de una sola GPU en un sistema multi-GPU
- Uso de varias GPU

2.4. Google Colaboratory

Otra alternativa es el uso de un entorno de desarrollo gratuito en la nube como Google Colaboratory, o "Colab", que permite a cualquier usuario escribir y ejecutar código arbitrario de Python en el navegador. Es especialmente adecuado para tareas de aprendizaje automático, análisis de datos y educación.

Algunas de las principales características de Colab son:

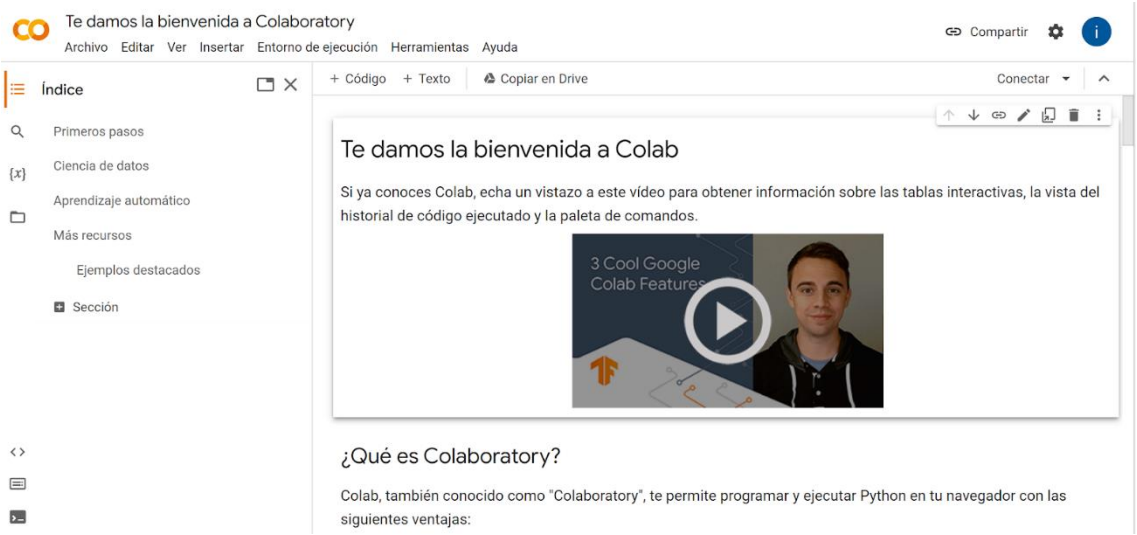
- Dado que se ejecuta en una máquina de Google, no es necesario realizar ninguna configuración.
- Google proporciona acceso gratuito a las GPU (limitadas).
- Es fácil de compartir, como cualquier archivo en el Google Drive.

Lo primero que se debe hacer para utilizar el Google Colab es acceder a la siguiente dirección URL en donde se da la bienvenida a Colab e indica los primeros pasos y algunos recursos disponibles, como se indica la Figura 13.

https://colab.research.google.com/notebooks/intro.ipynb

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Figura 13. Google Colab



En este punto siéntase libre de explorar el entorno de Colab teniendo una conexión a Internet.

Trabajo autónomo:

Cree y ejecute un script básico de Python (creado anteriormente) en Colab y Spyder/VS Code/Jupyter Notebook.

2.5. Conjuntos de datos disponibles para ciencia de datos

Existe una gran variedad de datasets públicamente disponibles para realizar tareas de análisis de datos. Algunos de ellos son los siguientes:

- UCI repo (https://archive.ics.uci.edu/)
- Kaggle (https://www.kaggle.com/datasets)
- Google dataset search (https://datasetsearch.research.google.com/)
- sklearn.datasets (https://scikit-learn.org/stable/datasets/toy_dataset.html)
- https://www.datosabiertos.gob.ec/
- https://datos.gob.es/es/
- etc.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Tamaños de datasets:

- **Dataset pequeño:** conjunto de datos del orden de un millar (alrededor de mil) de ejemplos de entrenamiento.

- **Dataset mediano:** conjunto de datos del orden de decenas de miles de ejemplos de entrenamiento.

- **Dataset grande:** conjunto de datos del orden de centenas de miles o millones de ejemplos de entrenamiento.

2.6. Algoritmos predictivos de clasificación

2.6.1. Regresión logística

Es un método de aprendizaje automático supervisado utilizado para resolver problemas de clasificación binaria o multinomial (más de dos clases). Aunque su nombre incluye la palabra "regresión", en realidad se trata de un algoritmo de clasificación.

Se utiliza comúnmente en una variedad de aplicaciones, como la detección de *spam* de correo electrónico, diagnósticos médicos, clasificación de documentos, entre otros. Por ejemplo, en medicina, es fundamental para predecir la probabilidad de ocurrencia de ciertas enfermedades, como la diabetes o el cáncer, basándose en factores de riesgo. En el sector financiero, se aplica en la evaluación del riesgo crediticio, determinando la probabilidad de que un cliente incumpla con sus pagos. En *marketing*, se utiliza para predecir la probabilidad de que un cliente compre un producto o responda a una campaña específica.

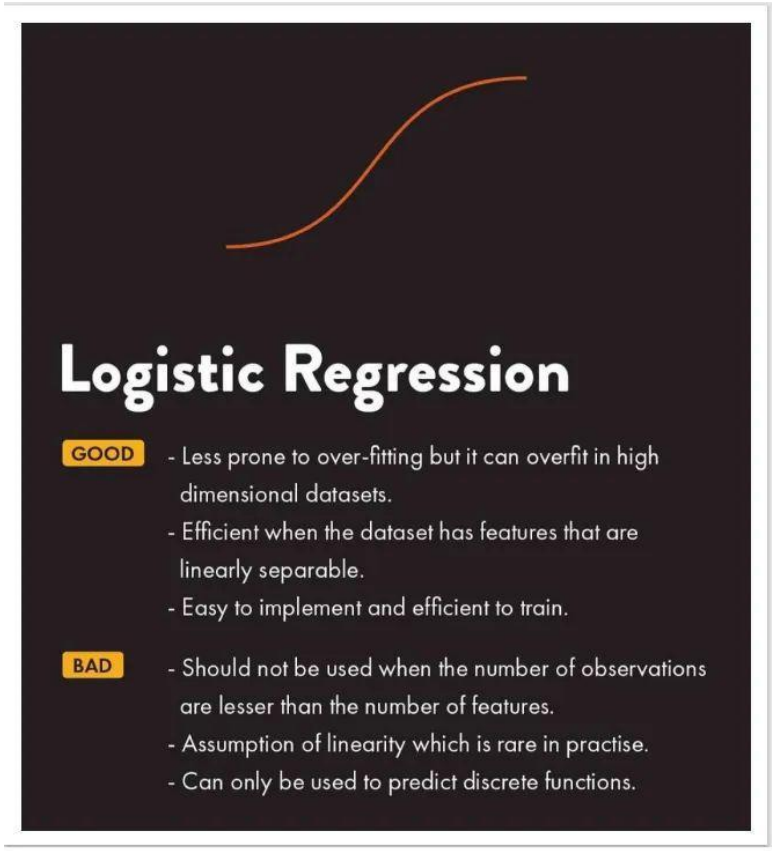
Además, la regresión logística es esencial en el campo de la investigación social y política para analizar la influencia de diversas variables sobre resultados binarios, como la decisión de votar por un candidato.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Su capacidad para proporcionar probabilidades y su facilidad de interpretación la hacen una herramienta valiosa para la toma de decisiones basada en datos.

En la Figura 14 se indican algunos pros y contras del algoritmo de regresión logística.

Figura 14. Pros y contras del algoritmo de regresión logística



Fuente: ML4Devs (2020)

2.6.2. Vecinos más cercanos (K-NN)

El algoritmo k-Nearest Neighbors (k-NN) es un método de aprendizaje automático supervisado utilizado para problemas de clasificación y regresión. El algoritmo k-NN (k-nearest neighbors) tiene diversas aplicaciones prácticas en campos como el análisis de datos, aprendizaje automático y sistemas de recomendación. En medicina, se utiliza para

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

clasificar pacientes en categorías de riesgo basándose en características similares a casos conocidos. En el sector financiero, ayuda en la detección de fraudes identificando patrones de transacciones sospechosas similares a casos previos de fraude. En el comercio minorista y en servicios de *streaming*, k-NN es esencial para los sistemas de recomendación, sugiriendo productos, películas o música basándose en las preferencias de usuarios con gustos similares. También es utilizado en la agricultura para la clasificación y diagnóstico de enfermedades de plantas, y en la robótica para la navegación y clasificación de obstáculos.

La simplicidad y efectividad del algoritmo lo hacen versátil para aplicaciones que requieren clasificación o recomendación basada en similitudes de datos.

En la Figura 15 indican algunos pros y contras del algoritmo K-NN.

Figura 15. Pros y contras del algoritmo K-NN

Fuente: ML4Devs (2020)

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Conclusiones

En esta unidad se ha revisado las herramientas (Colab, Spyder, VS Code, Jupyter notebook, etc.) y conjuntos de datos (UCI Repo, Kaggle, sklearn.datasets, etc.) para ciencia de datos, así como dos algoritmos predictivos de clasificación, regresión logística y vecinos más cercanos (K-NN). Los algoritmos de clasificación tienen un sin número de aplicaciones prácticas en diversos campos. En el sector financiero, se utilizan para evaluar la solvencia crediticia de los clientes, mientras que, en el ámbito de la salud, ayudan a diagnosticar enfermedades y a predecir resultados de tratamientos médicos. En el comercio electrónico, facilitan la recomendación personalizada de productos a partir del comportamiento de compra de los usuarios. Además, en la seguridad informática, contribuyen a la detección de actividades fraudulentas y en el mantenimiento predictivo, permiten prever fallos en equipos industriales, optimizando así las operaciones y reduciendo los costes. Estos algoritmos son valiosos por su capacidad de interpretar grandes volúmenes de datos y tomar decisiones informadas, lo que los hace esenciales en la era del Big Data. Sin embargo, para obtener un mejor rendimiento de cada uno de ellos es importante experimentar con los parámetros de cada algoritmo. Además, hay que considerar las ventajas y desventajas que presentan cada uno para la selección adecuada de un algoritmo de clasificación.

Bibliografía:

Berzal, Fernando (2019). *Redes neuronales & Deep Learning*. USA: Independently published.

Brownlee, J. (2023). *Machine Learning Mastery*.
https://machinelearningmastery.com/

García-Santillán, I. (2023). *Ciencia y analítica de datos*. Maestría en Inteligencia Artificial aplicada. Universidad Hemisferios.

InstitutoEmprende (2023). *Cómo crear un árbol de decisión*.
https://institutoemprende.com/como-crear-un-arbol-de-decisiones/

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Lara, J. (2014). *Minería de Datos.* Madrid: CEF-Udima

MathWorks (2023). *Machine Learning in Matlab.*
https://la.mathworks.com/help/stats/machine-learning-in-matlab.html

MaximaFormacion (2023). *Qué son los árboles de decisión y para qué sirven.* https://www.maximaformacion.es/blog-dat/que-son-los-arboles-de-decision-y-para-que-sirven/

ML4Devs (2020). *An Engineer's Trek into Machine Learning.*
https://www.ml4devs.com/articles/machine-learning-intro-for-developers/

Sierra, B. (2006). *Aprendizaje automático: conceptos básicos y avanzados.* Madrid: Prentice-Hall.

Torres, J. (2020). *Python Deep Learning: Introducción práctica con Keras y TensorFlow 2*. Marcombo.

UAB. (2016). *Curso de Detección de objetos.* Universidad Autónoma de Barcelona.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Aprendizaje automático

01 Herramientas para Ciencia de Datos: Python, Colab, Spyder, VS Code

02 Frameworks y librerías: TensorFlow, Keras, Pandas, Sci- Learn.

03 Algoritmos clásicos: Regresión logística, K-NN

04 Conjuntos de datos públicos: UCI, Kaggle

05 Aprendizaje supervisado y no supervisado

06 Algoritmos clásicos: árbol de decisión, Random Forest